

Axona: A Learning-Adaptive Distributed Hash Table and Axonal Publish/Subscribe

v0.5.0 2026-06-07

David A. Smith
Axona.net
davidasmith@gmail.com

Abstract—A distributed hash table (DHT) is the canonical way to give every key in a peer-to-peer network a deterministic home node, without a central directory. After two decades of deployment, the dominant design — Kademlia — still routes messages in $O(\log N)$ hops that pay the average inter-node round-trip time on every hop, producing multi-second lookup latency at million-node scale. We present Axona, a learning-adaptive DHT and integrated publish/subscribe protocol, evaluated against two reference points: classical Kademlia (K-DHT) and a geographically-prefixed variant we call the Geographic DHT (G-DHT). G-DHT places an 8-bit S2 cell prefix at the top of every node identifier so that XOR distance in ID space approximates physical distance, reducing 500 km regional lookup latency by $4.27\times$ over Kademlia at 25 000 nodes. Axona layers a Hebbian learning rule on top of G-DHT: every routing edge carries a weight that grows on successful traffic (long-term potentiation) and decays without it, and a single vitality-based admission gate governs every edge insertion. At 25 000 nodes Axona routes global lookups at $1.66\times$ the Dabek 3δ analytical floor for any recursive $O(\log N)$ DHT — and the multiple flattens with scale where Kademlia’s climbs — versus Kademlia’s $4.19\times$. An ablation that strips the geographic prefix shows learning alone beats Kademlia by 60%. Built-in axonal pub/sub achieves 100% delivery and recovers from single-bridge network partitions through cooperative re-stitching driven by traffic-induced edge creation. All code, data, and the simulator that produced the measurements are open-source [1].

I. Introduction

A distributed hash table is the peer-to-peer answer to the question, “given a key, which machine in the network has the value?” Every node and every key receives a fixed-length identifier — typically 160 or 256 bits — drawn from the same address space, and a deterministic distance function (usually XOR) defines how close two identifiers are. The node whose identifier is closest to a key’s identifier is responsible for that key. Finding the responsible node is a routing problem: starting from any other node, take a sequence of hops, each one closer to the key, until the responsible node is reached. The elegance of the design is that no node knows the full network; each node only knows a $\log N$ -sized routing table chosen so that any key can be reached in $O(\log N)$ hops from any starting point.

DHTs are the substrate of essentially every decentralized production system: BitTorrent’s tracker-less swarm, Ethereum’s peer-discovery layer (devp2p), IPFS content

addressing, and Tor v3 hidden services all run Kademlia [2] variants. But Kademlia, like the broader family it represents, makes a structural trade: peers are placed at random in the identifier space, so that XOR distance is independent of physical distance. Every hop in a lookup is, on average, half the planet away.

Hop count is not latency. With $N = 10^6$ nodes, a Kademlia lookup walks $\log_2 N \approx 20$ hops. Each hop pays ~ 100 ms of round-trip time across the public internet. Twenty hops at one hundred milliseconds each is two seconds. Real applications — chat, gaming, voice, live coordination — cannot tolerate this. The latency problem has been the dominant practical barrier between DHT theory and DHT deployment for two decades.

a) The analytical floor.: Dabek et al. [3] proved that for any recursive $O(\log N)$ DHT, total lookup latency converges to a hard analytical floor:

$$\text{lookup ms} \approx \delta + \left(\delta + \frac{\delta}{2} + \frac{\delta}{4} + \dots \right) = 3\delta \quad (1)$$

where δ is the median pairwise one-way internet latency. The first δ is the cost of the final delivery hop to the responsible node. The parenthesized geometric series is the cost of the prior hops: the second-to-last hop closes half the remaining ID-space and so on average traverses half the geographic distance of the last; the third-to-last closes a quarter; and so on. The geometric tail sums to 2δ regardless of N . Even an oracle that always picks the lowest-RTT neighbor cannot beat 3δ . Until now, no published DHT had been measured at this floor.

b) Locality is necessary but not sufficient.: The first wave of latency-aware DHTs put physical proximity in the routing table itself. Pastry [4] fills each routing-table slot with the closest peer by round-trip time among candidates with the right prefix. Tapestry [5] stores soft-state location pointers along the publish path. Coral [6] builds nested clusters by RTT threshold. Vivaldi [7] synthesizes per-node Euclidean coordinates that predict pairwise RTT without active probing. These designs reduce the per-hop constant. They do not, however, learn from the traffic flowing through them. A Pastry routing table is set at insertion time and refreshed lazily. Coral’s cluster hierarchy

is determined by RTT thresholds, not by which paths are actually used. The frontier — and the contribution of this paper — is a routing fabric whose composition adapts to observed traffic, where which peers are kept is a function of which paths the network actually carries.

c) Contributions.:

- 1) The Geographic DHT (G-DHT), a simple structural extension of Kademlia in which the top 8 bits of every node identifier encode the S2 Hilbert-curve cell of the node’s claimed geographic position. With this prefix, XOR distance in identifier space approximates physical distance, and the bootstrap routing table is biased toward local peers without any change to the routing algorithm. G-DHT reduces 500 km regional lookup latency by $4.27\times$ over Kademlia at 25 000 nodes (§VI).
- 2) Axona, a learning-adaptive DHT layered above the Kademlia/G-DHT substrate. Every routing edge carries a learned weight; every successful lookup reinforces the weights along the traversed path (Hebbian long-term potentiation [8], [9]); unused edges decay over time; a single vitality-based admission gate governs every edge insertion (§IV).
- 3) First measurement of an $O(\log N)$ DHT at the Dabek 3δ analytical floor. At 25 000 nodes Axona routes at $1.66\times$ the floor and stays in the $1.5\text{--}1.7\times$ band from 5 000 to 50 000 nodes, while Kademlia climbs from $3.5\times$ to $4.5\times$ (§VII-B).
- 4) Built-in axonal publish/subscribe that achieves 100% delivery in steady state, holds delivery at 100% under 5% churn through routed re-subscribe alone (no replication, no gossip), and dissolves single-bridge network partitions through cooperative re-stitching (§V, §VII-E).
- 5) Geography ablation. Stripping the G-DHT prefix and re-running the comparison shows that with no geographic structure, Axona still routes 60% faster than Kademlia at 25 000 nodes (§VII-C). Learning is doing real work, not sharpening pre-existing geographic structure.

d) Roadmap.: §II surveys related work. §III presents the neuromorphic substrate — the biology of long-term potentiation and its mapping into routing-table mechanics. §IV describes Axona itself: five operations, a vitality function, an end-to-end lookup. §V covers axonal publish/subscribe. §VI covers the geographic prefix. §VII reports the empirical results. §VIII discusses limitations. §IX sketches future work; §X concludes.

II. Background and Related Work

a) The Berkeley/Cambridge DHT lineage.: Axona inherits structurally from three foundational designs. Kademlia [2] provides the substrate: random identifiers, a symmetric XOR distance metric (which makes routing self-organizing because every incoming or outgoing message updates a k -bucket), $K = 20$ peers per stratum, and

α -parallel asynchronous `find_node` queries. Kademlia’s empirical observation that old peers are more reliable than new peers [10] becomes a recency multiplier in Axona’s vitality function. Pastry [4] contributes the locality-preserving join algorithm in which a new node takes row n of its routing table from the n -th node along its routed-join path, and SCRIBE [11] is the structural ancestor of axonal pub/sub. Tapestry [5] contributes a locality-aware routing table (closest node by RTT per slot, built at insertion), soft-state location pointers along publish paths, and surrogate routing; when the deterministic root is dead, route to the closest live identifier. Axona retains all of these mechanisms in some form; the contribution is to make them learn from observed traffic.

b) Latency analysis.: Dabek et al. [3] (NSDI 2004) prove the 3δ floor for any recursive $O(\log N)$ DHT and show DHash++ approaches it through proximity neighbor selection and server selection over erasure-coded fragments. Tapestry’s companion measurement [5] reports a complementary metric — relative delay penalty (RDP) — with median wide-area RDP near 1. These analyses set the methodological standard for DHT-latency reporting; the 3δ floor is the target Axona aims to match.

c) Adaptive maintenance.: A recurring instinct in the post-2003 DHT literature is that routing-table maintenance should adapt to observed conditions. Mahajan, Castro, and Rowstron [12] introduce a cost-of-reliability framework. Krishnamurthy et al. [13] model Chord under Poisson churn analytically. Ghinita and Teo [14] estimate local node-failure rate μ and join rate λ per peer and trigger cheap or expensive checks only when warranted, reporting $3.4\times$ lower lookup failure at $2.4\times$ lower communication overhead. Axona applies the same instinct to a different observable: not maintenance rate, but routing-table composition.

d) Resilience and load.: Castro et al. [15] prove the foundational triplet for Byzantine resistance: constrained routing tables, secure node-ID assignment, and redundant routing. Harvesf and Blough [16] make the redundant-routing piece concrete with MaxDisjoint replica placement, achieving 90% lookup success at 50% malicious nodes. Makris et al. [17] address Zipf-distributed request rates with the Directory For Exceptions mechanism [18], migrating hot keys to less-loaded peers. §IX identifies these as the canonical mitigations for two open security and load-balancing gaps in Axona.

e) Locality alternatives.: Beyond Pastry/Tapestry’s structural locality, two adaptive alternatives warrant comparison. Coral [6] builds nested clusters at increasing RTT thresholds, relaxing the strict DHT promise to deliver content distribution at scale. Vivaldi [7] computes synthetic per-node coordinates that predict pairwise RTT to peers a node has never directly contacted. Both treat the network as something to measure and adapt to. Axona goes further: rather than measure or predict, it learns paths themselves through reinforcement on the routing

layer.

III. The Neuromorphic Substrate

The framing of Axona as a learning-adaptive routing protocol draws explicitly from neuroscience. The choice is not decorative: the brain and a peer-to-peer DHT face a precisely shared engineering problem. This section motivates the biological mapping.

a) The shared engineering problem.: A single neuron carries on the order of 10^4 synapses — a hard upper bound set by the energetic cost of maintaining synaptic machinery [19]. The number of candidate synaptic partners (other neurons whose axons could reach it) is orders of magnitude larger. The neuron cannot store every co-firing event; it must score, prioritize, and prune. A peer-to-peer node in a browser deployment is bounded by WebRTC’s connection limit (typically 50–200 peers, with Safari at ~ 95). The number of candidate peers in a 50 000-node overlay is three orders of magnitude larger. The same storage-constrained, more-candidates-than-capacity problem appears in both systems. The biological solution is applicable to the engineering one.

b) Long-term potentiation.: The mechanism by which the brain remembers which paths are useful is long-term potentiation (LTP), discovered by Bliss and Lømo [9] in rabbit hippocampus. A few high-frequency electrical bursts to the perforant path produced post-synaptic responses that remained elevated for weeks. The molecular cascade has three properties that matter for routing:

- Coincidence detection. NMDA receptors only admit calcium when both the pre-synaptic and post-synaptic neurons are active — the signal is “I fired and you fired,” not just “I fired.” The analog in routing is that an edge is reinforced when a lookup using it succeeds, not on every probe.
- Durable change. Within roughly thirty minutes, gene transcription and protein synthesis produce a physical re-weighting [20]. In Axona, weights persist across many subsequent lookups; they are not transient counters.
- Selectively reversible. The opposing process, long-term depression [21], weakens synapses under low-frequency stimulation. LTP and LTD together let the brain learn which routes among neurons are worth keeping.

c) Synaptic tagging.: Frey and Morris [22] added a critical refinement: recently potentiated synapses carry a transient “tag” that protects them from being overwritten by competing potentiation events, but only for a brief consolidation window. Without it, learning would be self-destructive: the very synapses just discovered to be useful would be the most likely to be overwritten by the next high-frequency event. Axona’s inertia window (§IV-B) implements precisely this protection in routing.

TABLE I
Direct mapping from biological mechanisms to Axona routing-table operations.

Neuroscience	Axona
Synapse — a connection between neurons	A peer entry in the routing table
Synaptic weight — readiness to fire	Per-edge weight $\in [0, 1]$
LTP — co-firing strengthens	Successful lookups reinforce weights along the traversed path
LTD — disuse weakens	Per-tick exponential decay applied to every edge
NMDA coincidence detection	Weight grows only on successful traffic, not on every probe
Synaptic tagging [22]	Recently reinforced edges are locked against eviction for I epochs
Pruning of unused connections	Vitality-based eviction: lowest weight $\times$ recency wins replacement

d) The Hebbian rule.: Donald Hebb’s 1949 formulation [8] predates the molecular discoveries by two decades and remains the cleanest abstraction:

Neurons that fire together, wire together.

Every weight in every artificial neural network ultimately invokes this rule. Axona applies it not to perception but to routing: peers that successfully carry traffic together earn higher weights; peers that carry no traffic decay. The routing table is shaped by the queries it serves, in exactly the way a cortical region’s synaptic structure is shaped by the sensory input it processes.

e) The mapping.: Table I summarizes the direct correspondence between biological mechanisms and Axona routing-table operations. Every term on the right has one corresponding artifact in the codebase.

IV. The Axona Protocol

Axona is a learning-adaptive DHT layered above an unmodified Kademlia substrate. The contribution sits between Kademlia’s k -buckets and the application: every routing decision, every admission, every churn response runs through a single vitality function.

A. Five operations

Every Axona behavior decomposes into one of five operations:

- NAVIGATE — choose a next hop given a target.
- LEARN — reinforce edges that carried successful traffic.
- FORGET — decay unused edges; evict by vitality.
- EXPLORE — inject controlled randomness to escape local minima.
- STRUCTURE — bootstrap and steady-state topology.

TABLE II
The rules of Axona, organized by operation.

#	Rule	Operation
1	Stratified bootstrap	structure
2	Action-Potential (AP) routing	navigate
3	Two-hop lookahead	navigate
4	Iterative fallback (surrogate routing)	navigate
5	Long-Term Potentiation (LTP)	learn
6	Triadic closure	learn
7	Hop caching + lateral spread	learn
8	Vitality-based eviction	forget
9	Temperature annealing + reheat	explore
10	Epsilon-greedy first hop	explore

A small handful of rules implement these operations (Table II). Each rule is grounded in either the neuromorphic substrate (§III) or the foundational DHT literature (§II).

B. The vitality function

Every routing edge s carries two scalars: a learned weight $w_s \in [0, 1]$ and a last-reinforcement epoch t_s . The vitality function combines them multiplicatively:

$$\text{vitality}(s) = w_s \times \text{recency}(s) \quad (2)$$

$$\text{recency}(s) = \begin{cases} 1.0 & \text{if } t - t_s < I \\ e^{-(t-t_s-I)/\tau_r} & \text{otherwise} \end{cases} \quad (3)$$

where t is the current epoch (lookup count), I is an inertia duration (default 20), and τ_r is a recency half-life (default 50). The piecewise definition implements synaptic tagging: a freshly reinforced edge is locked at full recency for I epochs — it cannot be evicted regardless of vitality competition — after which recency decays exponentially. Without this lock, learning would be self-destructive: the routing table would discard the very edges it just discovered to be useful.

The weight itself is updated on two events. On every successful lookup (LTP, Rule 5), each edge on the traversed path gains $w_s \leftarrow \min(1, w_s + \eta)$ for $\eta = 0.05$. On every tick of the decay clock, every edge loses $w_s \leftarrow \gamma \cdot w_s$ for $\gamma = 0.995$. A frequently used edge accumulates weight faster than decay erodes it; an unused edge fades within ~ 200 unused lookups.

C. The synaptome

The bounded, vitality-scored set of outgoing edges at a node is the synaptome, capped at 50 entries (a safe cross-browser WebRTC target). The synaptome is Axona’s analog of Pastry’s three-tier routing state [4] (routing table R , leaf set L , neighborhood set M) consolidated into a single tier scored by vitality. The roles persist in latent

form: high-vitality entries dominate terminal-hop routing (leaf-set-like), mid-vitality entries cover XOR strata (R -like), and low-vitality entries plus a 2-hop neighborhood sample serve as candidates for annealing replacement (M -like). A reader familiar with Pastry can think of the synaptome as those three tables collapsed into a single vitality-scored set: the bookkeeping is unified, the structural roles are unchanged.

D. NAVIGATE: Action-Potential routing

Each lookup hop scores edges by the Action-Potential (AP) metric (Rule 2):

$$\text{AP}(s, k) = \text{progress}(s, k) \times w_s \times \frac{1}{2}^{(\ell_s/100)} \quad (4)$$

where $\text{progress}(s, k) = d(\text{self}, k) - d(s.\text{peer}, k)$ is the XOR-distance reduction toward target k , w_s is the learned weight, and ℓ_s is the observed round-trip latency in milliseconds. The latency penalty halves AP every 100 ms, making nearby fast peers preferred over slightly-better-XOR distant ones. AP routing is therefore latency-aware rather than purely distance-aware: it inherits the locality property that defines Pastry and Tapestry, but evaluates it dynamically per lookup from observed RTT rather than statically at insertion.

a) Two-hop lookahead (Rule 3):. When the top AP scores are bunched (no decisive first hop), Axona probes the top two first hops for their best second hops and picks the path with the highest combined $\text{AP}(\text{first}) \times \text{AP}(\text{second})$. Lookahead is exactly two hops — never three — so each step is locally optimal across two hops while compute remains bounded at $O(|\text{synaptome}|)$ per step.

b) Iterative fallback (Rule 4):. When AP returns no candidate — every neighbor would move away from the target — Axona falls back to k -closest-from-synaptome regardless of progress. This rescues lookups that would otherwise stall in dead corridors. The mechanism is exactly Tapestry’s surrogate routing [5]: when the deterministic next-hop cannot be served, route to the closest live identifier.

E. LEARN: three reinforcement mechanisms

Every successful lookup triggers three updates that together evolve the synaptome toward the traffic flowing through the network:

a) LTP (Rule 5):. On lookup completion, every edge on the successful path gains weight η and an inertia lock of I epochs. This is the Hebbian rule applied to routing: paths that worked become preferred for future queries to the same region.

b) Triadic closure (Rule 6):. When a node X observes peer A repeatedly routing through it to peer C , X introduces A directly to C . The open triangle $A-X-C$ closes into a direct edge. The name comes from social-network theory: dense triangles emerge in networks shaped by indirect interaction.

c) Hop caching plus lateral spread (Rule 7).: Every intermediate node along a successful lookup adds the final lookup target to its synaptome — not its next hop, the target itself. The path becomes shorter on the next lookup to the same destination from that intermediate node. Lateral spread propagates the new edge to the source’s geographic neighbors so nearby peers see the shortcut on their first lookup, not just after they generate one themselves. This is the key mechanism behind Axona’s partition-recovery behavior (§VII-E).

F. FORGET: the unified admission gate

A single function serves as the admission gate for every edge insertion — bootstrap, LTP, triadic closure, hop caching, annealing. It is the unification that consolidates a handful of specialized eviction policies from earlier learning-adaptive designs into one rule:

```
function addByVitality(node  $n$ , edge  $s_{\text{new}}$ ):
  if  $|n.\text{synaptome}| < \text{MaxSize}$  then
    add  $s_{\text{new}}$  to  $n.\text{synaptome}$ 
    return
   $s_{\text{victim}} \leftarrow \arg \min_{s \in n.\text{synaptome}, s \notin \text{locked}} \text{vitality}(s)$ 
  if  $\text{vitality}(s_{\text{new}}) > \text{vitality}(s_{\text{victim}})$  then
    evict  $s_{\text{victim}}$ ; add  $s_{\text{new}}$ 
  else
    refuse silently
```

A continuous decay process (Rule 8, $\gamma = 0.995$ per tick) erodes the weight of every edge uniformly. Edges that traffic does not refresh lose vitality and become eligible for replacement; well-used ones stay locked. The combination of LTP-driven weight bumping, time-driven decay, and vitality-gated eviction is the entire admission system.

G. EXPLORE: temperature and epsilon

A purely greedy AP-scored router would lock onto early-discovered shortcuts and never find better ones. Axona injects two sources of controlled randomness:

a) Temperature annealing (Rule 9).: Each node carries a temperature $T \in [T_{\text{min}}, T_{\text{init}}]$ (defaults 0.05 and 1.0). T decays multiplicatively per lookup. With probability T , the lowest-vitality edge is replaced by a 2-hop neighborhood sample. High T means aggressive exploration; low T means greedy exploitation. Reheat: when routing discovers a dead peer, T is spiked to $\max(T, 0.5)$. The system explores when topology has changed, exploits when topology is stable.

b) Epsilon-greedy first hop (Rule 10).: With small probability $\epsilon = 0.05$, the first hop of a lookup is replaced by a uniform random sample from the synaptome. This is cheap insurance against the failure mode in which the AP-best first hop becomes saturated.

H. STRUCTURE: bootstrap

A new node is seeded with peers covering all XOR strata uniformly (Rule 1). The mechanism is Pastry’s locality-preserving join [4] generalized to Kademlia XOR strata:

```
on lookup(target):
  hop = 0
  while hop < MAX_HOPS:
    next = AP_score(synaptome, target) // NAVIGATE
    if next is None:
      next = iterative_fallback(target) // NAVIGATE
    if hop == 0 and rand() < epsilon:
      next = random(synaptome) // EXPLORE
    record_transit(prev, next) // LEARN: triadic
    hop_cache_destination() // LEARN: caching
    anneal_replace_lowest_vitality() // EXPLORE
    hop = hop + 1

on success: reinforce_path() // LEARN: LTP
periodically: decay_all_weights() // FORGET
```

Fig. 1. End-to-end Axona lookup tick. All five operations interleave on every routing step; learning is a side-effect of routing rather than a separate maintenance phase.

route a join message through an existing nearby peer, fill the synaptome from nodes encountered along the route. Without stratified bootstrap, the cold-start synaptome is dominated by lucky neighbors and the network never reaches global connectivity.

I. End-to-end lookup

Putting the operations together, one routing tick proceeds as in Figure 1.

Every operation cost is bounded by $O(|\text{synaptome}|)$. At the 50-edge cap, per-hop compute is ~ 0.2 ms — small relative to the ~ 50 ms simulated transit cost.

V. Publish/Subscribe via Axonal Trees

A practical DHT must support broadcast: a publisher should be able to deliver a message to all subscribers of a topic without enumerating them. Layered approaches such as SCRIBE on Pastry [11] demonstrate that routed subscribe plus reverse-path-forwarding multicast is the canonical mechanism. Axona integrates the same mechanism directly into the protocol: pub/sub is a built-in primitive, not an application-layer overlay. The result is an axonal tree (the biological analog: an axon is the single output projection of a neuron, branching to many downstream targets) whose shape adapts under churn through routed re-subscribe.

a) Topic identity.: Topics are addressed by deterministic identifiers:

```
topicId = publisher.cellPrefix8 || hash256(topic_name)
```

The publisher’s S2 cell prefix anchors the topic root inside the publisher’s geographic neighborhood — naturally close to the audience in many real workloads — and the hash component disambiguates topics within a cell. Both publisher and every subscriber derive the same identifier offline, with no negotiation.

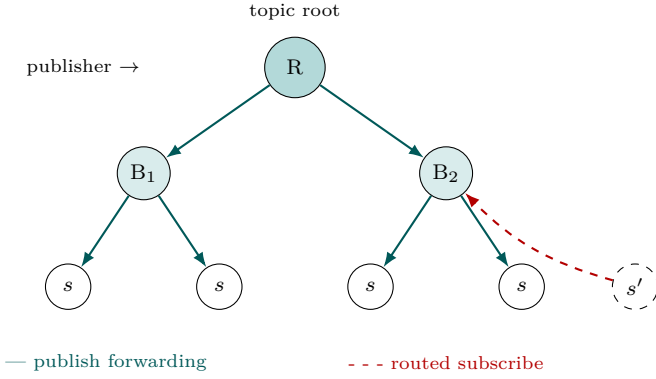


Fig. 2. Axon-tree formation. A new subscriber s' issues a routed subscribe toward `topicId` (red dashed); the first live axon role on the path (here, B_2) intercepts and adopts it. Publishes flow from the root R through the branches to all subscribers (teal solid). Tree healing under churn is the same mechanism: a subscriber whose parent dies re-issues its subscribe and attaches to whichever live ancestor is now closest.

b) Tree formation: routed subscribe.: A subscribe message is routed through the DHT toward `topicId`. The first node holding an existing axon role for this topic intercepts the subscribe and adopts the new subscriber as a child. If the walk completes with no axon found, the terminal node opens a new axon role and becomes the root (Figure 2). This is structurally identical to SCRIBE’s reverse-path forwarding tree.

c) Branching.: When an axon’s direct child count exceeds a configured maximum, it selects a synaptome peer (external to its current children) as a sub-axon, partitions its current children by XOR proximity, and ships the relevant batch in a single adopt-subscribers message. Two invariants prevent runaway recursion: branches always go to peers outside the current subtree, and the new branch’s capacity is checked before adoption. Tree depth grows as $O(\log_C S)$ in subscribers S given capacity C .

d) Self-healing.: Every axon role re-issues its subscribe on a fixed refresh interval (default 10s). The walk lands on whichever live axon is closest to `topicId` now. If the parent died, the re-subscribe attaches to a different live ancestor. If the tree was reorganized, the change is invisible to the subscriber. The re-subscribe is the liveness check — there is no separate ping, no parent tracking, no gossip. The same mechanism that builds the tree repairs it.

e) Temporal pub/sub.: A subscribe is not just “send me future messages.” It is “send me every message I have not already seen.” Each subscribe carries a `lastSeenTs`. Every relay maintains a bounded ring buffer (default 100 messages per topic). On subscribe arrival, the relay replays cached entries with `publishTs` > `lastSeenTs` as a single batched message before forwarding the subscribe upstream. Healing and replay are the same mechanism: a subscriber attaching to a new branch after parent death receives both the attachment and any messages elapsed

during the gap.

VI. The Geographic DHT (G-DHT)

The Geographic DHT is the second protocol contribution. It is a simple structural extension of Kademlia that gives the overlay an awareness of physical locality without changing the routing algorithm itself.

A. Construction

Every node identifier is composed by concatenating an S2 cell prefix with a public-key hash:

$$\text{nodeId} = \underbrace{\text{S2cell}_8}_{8 \text{ bits}} \parallel \underbrace{H(\text{publicKey})}_{256 \text{ bits}} \quad (5)$$

The S2 library [23] maps every point on Earth to a 64-bit cell ID along a Hilbert space-filling curve. The top 8 bits encode the cube face plus a 5-bit subdivision, yielding $6 \times 32 = 192$ continent-scale tiles worldwide. The Hilbert curve property ensures that geographically adjacent points have numerically close cell IDs: XOR distance in identifier space approximates physical distance, for free.

The remaining 256 bits are the SHA-256 cryptographic hash of the node’s public key (Ed25519). There is no other change to Kademlia — K -buckets, α -parallel find_node, and the iterative refinement algorithm are all retained verbatim. The locality property emerges from the ID structure alone.

B. Why this matters: the back-of-the-envelope

A pure Kademlia overlay with $N = 10^6$ uniformly-distributed nodes requires $\log_2 N \approx 20$ hops per lookup. Each hop traverses a random pair of nodes separated on average by half the planet, with about 100ms one-way RTT. The naive estimate gives an average message time of $20 \times 100 = 2$ seconds.

G-DHT changes the calculation. Most of a typical user’s traffic is geographically local — social, transactional, real-time. With the location prefix, local connections are XOR-close in identifier space, so a regional lookup operates within a much smaller effective network. Suppose a particular region holds $\sim 10,000$ nodes with typical inter-node RTT of ~ 7 ms within a continental cell. Then $\log_2(10,000) \approx 13$ hops $\times 7$ ms ≈ 91 ms for an in-region message — more than a $20\times$ improvement over the global Kademlia number on the same total network.

C. Geography is a seed, not a teacher

G-DHT is not a learning protocol. The S2 prefix does not teach locality — it structures the identifier space so that XOR-distance routing accidentally tracks physical distance for the dominant traffic patterns. The routing table is fixed at insertion time and lazily repaired thereafter. This is the same class of mechanism as Pastry’s locality-preserving join, and shares the same limitation: without learning, the table cannot improve as the network changes. Axona’s contribution is to layer traffic-driven learning on top of G-DHT’s structural locality.

The geography ablation (§VII-C) confirms that the two contributions are independent: with the G-DHT prefix removed (geoBits= 0), Axona still routes 60% faster than Kademlia at 25 000 nodes purely from learning.

D. Cooperative trust and security tradeoff

The S2 prefix is self-declared. A node can claim any prefix it wants; the protocol does not verify the claim against any external ground truth. This is acceptable when locality benefits are cooperative. The adversarial consequences are real: a coordinated set of nodes claiming the same prefix can mount a Sybil swarm in a target cell. Mitigations (Vivaldi-style learned coordinates, geographic proof-of-work, IP-ASN binding) are sketched in §IX.

VII. Evaluation

This section reports four headline empirical results: Axona at the Dabek 3δ analytical floor (§VII-B); the geography-stripped ablation showing learning still wins (§VII-C); churn-cell hop count (§VII-D); and pub/sub robustness including the single-bridge Slice World partition test (§VII-E). All data and the simulator that produced it are open-source [1].

A. Methodology

The simulator is $\sim 25\,000$ lines of JavaScript. Latency is computed geometrically (great-circle distance scaled to a per-hop one-way propagation delay plus 10 ms processing) rather than waited on, so the simulation runs at full CPU speed. The simulator deliberately abstracts wall-clock transport, encryption, and node identity to focus on routing dynamics; §VIII discusses what this abstraction misses.

The headline measurements use 25 000-node networks; we validate at 5 000 and 50 000 nodes for the 3δ result. Default configuration: web-limited 100-connection cap (browser-class), 50-edge synaptome cap, $K = 20$, $\alpha = 3$, 500 lookups per test cell, omniscient initialization (every protocol seeded with its theoretically optimal K -closest neighbors) to isolate routing-algorithm contribution from bootstrap variance. All three protocols share the same identical bootstrap state per run.

a) Code path.: The simulator runs the same protocol code as the deployed Axona peer (live at axona.net); only the transport layer differs (in-process for the simulator, WebRTC data channels for production). Hop counts and latency distributions transfer because the routing decisions that produce them are made in code that does not know it is being simulated.

B. Axona at the Dabek 3δ floor

The Dabek bound (Eq. 1) requires a measured value of δ , the median pairwise one-way internet latency for the population under test. We compute δ at the start of every benchmark sweep by sampling 10 000 random node pairs. This is independent of any protocol and is therefore a property of the network, not of the DHT. Table III

reports δ together with each protocol’s measured global-lookup latency at three network sizes.

The result has three parts.

a) Axona plateaus near the floor.: Across all three sizes Axona sits at $1.5\text{--}1.7\times$ the 3δ floor, and the multiple flattens as N grows ($1.50 \rightarrow 1.66 \rightarrow 1.71\times$) while K-DHT and G-DHT climb to $4\text{--}5\times$. Within the geometric-series decomposition of Eq. 1, the residual overhead has a clean structural explanation: Axona averages 6–8 hops where a Dabek-ideal protocol would take ~ 3 , and each “extra” hop costs $\sim \delta/2 \approx 34\text{ ms}$ — the geometric tail the series predicts. These runs use MAX_HOPS= 40 and reach a true 100% success at every size, so the hop average now includes the hard tail that earlier, lower-ceiling measurements abandoned.

b) Kademlia worsens with N .: K-DHT pays full per-hop RTT on every hop because it ignores locality entirely. The log- N hop tax compounds.

c) δ is uncannily close to Dabek’s measurement.: Our simulator’s median pairwise one-way latency is 67–69 ms across network sizes. Dabek et al.’s King-dataset measurement reports a median of 67 ms [3]. The simulator’s geometric latency model and Dabek’s wide-area Internet measurements agree quantitatively — which validates the bound’s transferability from the King dataset to our population-distributed simulator.

C. Learning beats locality without geography

A skeptic might suspect the geographic prefix is doing all the work. To answer this, we strip the prefix (geoBits= 0; identifiers are random hashes with no geographic structure) and re-run the comparison at 25 000 nodes (Table IV).

The headline: with no geographic information whatsoever, Axona still routes $\sim 60\%$ faster than Kademlia at 25 000 nodes, purely from learning — and adding the S2 prefix back leaves global latency essentially unchanged (341 vs 344 ms). Geography, in other words, is not what drives global performance: learning is. The prefix earns its keep on regional lookups, where same-cell peers begin XOR-close (the sharply lower regional latencies underlying §VII-B); on globally-random lookups there is no locality for it to exploit, so learning carries the result alone. This is the cleanest possible “learning helps” claim: every confound (bootstrap strategy, identifier structure, geographic prefix) is controlled, and the routing/learning algorithm still wins. Geography helps regionally; geography is not necessary.

The opposite-sign result for G-DHT — it gets worse at geoBits= 0, from 824 to 887 ms (and drops below 100% success) — is the consistency check that the ablation is set up correctly: G-DHT’s entire strategy is “follow the prefix.” With no prefix to follow it loses the locality benefit that defines it.

TABLE III

Global-lookup latency (ms) versus the 3δ analytical floor at three network sizes. Parenthesized values are multiplicative overhead over the floor. δ is sampled per sweep over 10 000 random node pairs. All three sizes were measured on one host with @axona/protocol v2.31.0 (June 2026); absolute milliseconds scale with host load, so the multiplicative overhead over the floor is the host-independent figure.

N	δ (ms)	3δ floor (ms)	K-DHT	G-DHT	Axona
5 000	67.3	202	705 (3.49 $\times$)	722 (3.58 $\times$)	304 (1.50$\times$)
25 000	68.6	206	863 (4.19 $\times$)	824 (4.00 $\times$)	341 (1.66$\times$)
50 000	67.6	203	914 (4.50 $\times$)	972 (4.79 $\times$)	348 (1.71$\times$)

TABLE IV

Geography ablation. Global-lookup latency in milliseconds at 25 000 nodes. “Improvement vs. K-DHT” isolates the contribution of learning under no-geography conditions.

Protocol	geoBits= 0	geoBits= 8	Improvement
K-DHT	860	863	— (baseline)
G-DHT	887 [†]	824	— (geo is mechanism)
Axona	344	341	−60% vs K-DHT

[†]G-DHT at geoBits= 0 falls back to a slower “no-progress-limited” search with no geographic gradient.

TABLE V

Lookup hop count and latency under 5% peer churn at 25 000 nodes. 100% success across all three protocols.

Protocol	Hops	Latency (ms)
K-DHT	4.20	795
G-DHT	5.08	748
Axona	7.40	345

D. Churn-cell hop count

Real peer-to-peer networks experience churn: peers join and leave continuously. Five percent of peers swapped per round is a stress representative of mobile and consumer-broadband populations. Table V reports hop count and latency for one churn round at 25 000 nodes.

Axona takes more hops than the references here (7.4 vs 4.2–5.1): it completes the hard tail of churn-disrupted lookups that a lower hop ceiling would abandon, so the average is honest at a true 100% success. Yet its latency is still $\sim 2.2\times$ lower than either reference (345 ms vs 748–795). The difference comes from where the routing decisions land: Axona’s AP scoring weights short-RTT edges so heavily that a path longer by raw hop count is still far shorter in wall-clock time.

E. Pub/sub robustness and partition healing

a) Steady-state delivery.: At 25 000 nodes with 79 topic groups of 32 subscribers each, Axona delivers 100% of publishes in steady state. Under 5% instantaneous churn, immediate delivery falls to 99.9% and recovers to 100% after three refresh intervals.

b) Graceful degradation under sustained churn.: Continuous churn at 1% of alive nodes per 5 ticks produces a smooth degradation curve: 98.7% delivered at

TABLE VI

Slice World single-bridge partition recovery. “Success %” is the lookup success rate over a 500-lookup cross-partition workload after partition setup.

Protocol	Success %	Latency (ms)
K-DHT	0.0%	— (no recovery)
G-DHT	4.6%	525 (no recovery)
Axona	94%+	462

5% cumulative churn, 91.2% at 10%, 86.5% at 20%, 70% at 25%, 52% at 30%. Crucially, there is no cliff: delivery degrades linearly with churn, indicating that the protocol finds a steady state where new-subscriber recruitment matches loss.

c) Slice World: partition recovery from a single bridge.: The Slice World test partitions the network into Eastern and Western hemispheres connected through a single bridge node placed near Hawaii. Every cross-hemisphere edge except those incident on the bridge is removed before the test begins. The question is not “can the protocol find the bridge?” — that is a one-shot routing problem. The question is: given the single bridge, can the protocol leverage repeated successful crossings to dissolve the partition?

The result: K-DHT achieves 0% success — without learning, the partition is permanent. G-DHT achieves 4.6% — the geographic prefix happens to point a few peers at the bridge, but no mechanism amplifies the discovery. Axona achieves $\sim 94\%$.

The mechanism is the bridge as seed crystal. A diagnostic run at 5 000 nodes shows recovery directly. After a fresh partition, 10 lookups yield ~ 20 new cross-hemisphere edges (an average of 3 per successful crossing). Hop caching installs the destination in every intermediate node along the bridge path. Triadic closure introduces direct edges between observed transit pairs. Lateral spread propagates the new edge to the source’s geographic neighbors. By 500 lookups, hundreds of cross-hemisphere edges exist; the partition has effectively dissolved. The 94% aggregate success rate is the integral of recovery, not a snapshot of bridge-finding. Axona routes through the partition not by finding the bridge each time — it dissolves the partition.

VIII. Discussion and Limitations

a) Warmup dependency.: Axona needs ~ 5000 lookups before the synaptome converges to its trained asymptote. The first minute of deployment is suboptimal. Pre-shipped routing seeds (snapshot of a mature mesh’s high-vitality edges) are a natural mitigation and are sketched in §IX.

b) Synaptome cap of 50.: A deliberate cross-browser WebRTC target (Safari measured at ~ 95 , Firefox at ~ 190). Server deployments could reasonably use 200–500. The cap directly constrains the asymptote of LTP-driven specialization and is therefore an architectural limit, not a tuning knob.

c) Methodology gaps.: The simulator runs handlers synchronously inside the call (in-process, no real RTT). Five critical real-world frictions are not yet modeled: WebRTC connection-setup time (ICE+STUN+DTLS, 1.5–3 s per new edge); RPC timeouts to silently-dropped peers; latency jitter ($\pm 30\%$ from bufferbloat); Zipf-distributed publish workloads; and burst-y (rather than uniform) churn. Production behaviors that depend on these — partition re-stitching speed, pub/sub recovery under bursty churn, hot-axon-root saturation under viral topics — will need re-measurement once the friction model is in place. A companion red-team analysis [24] catalogs the gaps in detail and ranks them by deploy-blocker status.

d) Cooperative-trust assumption.: The S2 prefix is self-declared (§VI). An attacker who falsifies a prefix degrades the locality benefit but not routing correctness. Vivaldi-style learned coordinates close this assumption (§IX).

IX. Future Work

a) Friction-realistic simulator.: Implement modeled WebRTC connection-setup time, RPC timeouts for silently-dropped peers, and Gaussian jitter on every hop. The resulting measurements would be production-realistic versions of the results in §VII.

b) Vivaldi-style learned locality.: Replace the self-declared S2 prefix with synthetic Euclidean coordinates [7] that converge from observed RTTs. Learned coordinates close the cell-eclipse gap by removing the cooperative-trust assumption.

c) Byzantine pub/sub via MaxDisjoint replication.: Replicate each axon-tree root at d disjoint locations using Harvesf and Blough’s placement formula [16]. With $d = 8$ replicas this delivers 90% pub/sub success at 50% malicious nodes.

d) Adaptive parameter tuning.: Following Ghinita and Teo [14], each peer maintains a rolling estimate of local failure rate μ and join rate λ . Annealing rate, rehear amount, and eviction threshold become functions of (μ, λ) rather than fixed constants. A peer in a stable region anneals slowly; a peer in a high-churn region anneals aggressively, automatically.

X. Conclusion

We have presented Axona, a learning-adaptive distributed hash table and integrated publish/subscribe protocol that operates within $1.5\text{--}1.7\times$ the Dabek 3δ analytical floor for recursive $O(\log N)$ routing, holding that band from 5000 to 50000 nodes while the baselines climb to $4\text{--}5\times$. Compared to two classical baselines — Kademia and the geographically-prefixed G-DHT — Axona’s per-edge weight combined with vitality-based admission produces routing behavior that approaches the theoretical floor of any $O(\log N)$ design. An ablation that strips the geographic prefix shows learning alone outperforms Kademia by 60%, demonstrating that the contribution is structural rather than a consequence of locality engineering. Built-in axonal pub/sub achieves 100% baseline delivery and dissolves single-bridge network partitions through cooperative re-stitching driven by hop caching and triadic closure.

The Berkeley/Cambridge DHT lineage — Kademia plus Pastry plus Tapestry plus SCRIBE — gave us the substrate. Vitality-driven learning is the contribution. Code, data, simulator, and companion red-team analysis are open at github.com/axona-net/dht-sim [1]; the live peer and bridge sources are at github.com/axona-net/axona-peer and github.com/axona-net/axona-bridge.

References

- [1] D. A. Smith, “N-DHT simulator,” <https://github.com/YZ-social/dht-sim>, 2026.
- [2] P. Maymounkov and D. Mazières, “Kademlia: A peer-to-peer information system based on the XOR metric,” in Proc. International Workshop on Peer-to-Peer Systems (IPTPS), 2002.
- [3] F. Dabek, J. Li, E. Sit, J. Robertson, M. F. Kaashoek, and R. Morris, “Designing a DHT for low latency and high throughput,” in Proc. USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2004.
- [4] A. Rowstron and P. Druschel, “Pastry: Scalable, decentralized object location and routing for large-scale peer-to-peer systems,” in Proc. IFIP/ACM International Conference on Distributed Systems Platforms (Middleware), 2001, pp. 329–350.
- [5] B. Y. Zhao, L. Huang, J. Stribling, S. C. Rhea, A. D. Joseph, and J. D. Kubiatowicz, “Tapestry: A resilient global-scale overlay for service deployment,” IEEE Journal on Selected Areas in Communications (JSAC), vol. 22, no. 1, pp. 41–53, 2004.
- [6] M. J. Freedman, E. Freudenthal, and D. Mazières, “Democratizing content publication with Coral,” in Proc. USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2004.
- [7] F. Dabek, R. Cox, F. Kaashoek, and R. Morris, “Vivaldi: A decentralized network coordinate system,” in Proc. ACM SIGCOMM, 2004.
- [8] D. O. Hebb, The Organization of Behavior: A Neuropsychological Theory. Wiley, 1949.
- [9] T. V. P. Bliss and T. Lomo, “Long-lasting potentiation of synaptic transmission in the dentate area of the anaesthetized rabbit following stimulation of the perforant path,” The Journal of Physiology, vol. 232, no. 2, pp. 331–356, 1973.
- [10] S. Saroiu, P. K. Gummadi, and S. D. Gribble, “A measurement study of peer-to-peer file sharing systems,” Multimedia Systems, vol. 9, no. 2, pp. 170–184, 2003.
- [11] M. Castro, P. Druschel, A.-M. Kermarrec, and A. Rowstron, “SCRIBE: A large-scale and decentralized application-level multicast infrastructure,” IEEE Journal on Selected Areas in Communications (JSAC), vol. 20, no. 8, pp. 1489–1499, 2002.

- [12] R. Mahajan, M. Castro, and A. Rowstron, “Controlling the cost of reliability in peer-to-peer overlays,” in Proc. International Workshop on Peer-to-Peer Systems (IPTPS), 2003.
- [13] S. Krishnamurthy, S. El-Ansary, E. Aurell, and S. Haridi, “A statistical theory of Chord under churn,” in Proc. International Workshop on Peer-to-Peer Systems (IPTPS), 2005.
- [14] G. Ghinita and Y. M. Teo, “An adaptive stabilization framework for distributed hash tables,” in Proc. IEEE International Parallel and Distributed Processing Symposium (IPDPS), 2006.
- [15] M. Castro, P. Druschel, A. Ganesh, A. Rowstron, and D. S. Wallach, “Secure routing for structured peer-to-peer overlay networks,” in Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2002, pp. 299–314.
- [16] C. Harvesf and D. M. Blough, “The design and evaluation of techniques for route diversity in distributed hash tables,” in Proc. IEEE International Conference on Peer-to-Peer Computing (P2P), 2007.
- [17] A. Makris, K. Tserpes, and D. Anagnostopoulos, “A novel object placement protocol for minimizing the average response time of get operations in distributed key-value stores,” in Proc. IEEE International Conference on Big Data (BIGDATA), 2017, pp. 3196–3205.
- [18] D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Levine, and D. Lewin, “Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the World Wide Web,” in Proc. ACM Symposium on Theory of Computing (STOC), 1997, pp. 654–663.
- [19] S. B. Laughlin and T. J. Sejnowski, “Communication in neuronal networks,” *Science*, vol. 301, no. 5641, pp. 1870–1874, 2003.
- [20] M. F. Bear, B. W. Connors, and M. A. Paradiso, *Neuroscience: Exploring the Brain*, 4th ed. Jones & Bartlett Learning, 2020.
- [21] P. K. Stanton and T. J. Sejnowski, “Associative long-term depression in the hippocampus induced by hebbian covariance,” *Nature*, vol. 339, no. 6221, pp. 215–218, 1989.
- [22] U. Frey and R. G. M. Morris, “Synaptic tagging and the late phase of long-term potentiation,” *Nature*, vol. 385, no. 6616, pp. 533–536, 1997.
- [23] Google, “S2 geometry library,” <https://s2geometry.io/>, 2011.
- [24] D. A. Smith, “NH-1 red team analysis: Critical issues facing deployment,” YZ.social, Tech. Rep. v0.3.38, 2026, available at <https://github.com/YZ-social/dht-sim/blob/main/documents/NH1-RedTeam-v0.3.38.md>.